

Study Guide — Module 3 online Evaluation (Attempt 1)

Contents

1. Foundations of Large Language Models	1
2. Prompt Engineering and Reasoning Strategies	3
3. Fundamentals of AI Agents and Tool Usage	5
4. Structured Outputs and Reliable Agent Workflows	6
5. Embeddings and Semantic Retrieval	8
6. Retrieval-Augmented Generation (RAG)	9
7. Productionizing LLM Applications with FastAPI	11
8. Orchestration and Agent Workflow Design with Langgraph	12
Coverage checklist	15

Batch: IITP-AIMLT-2601 · **Program:** Certificate Program in Artificial Intelligence and Machine Learning

This guide compiles the key concepts you need for your upcoming evaluation. Work through each section in order — it builds from LLM foundations up to full production systems. Focus on the worked examples and definitions, and use the **Key terms** line at the end of each section to self-test.

1. Foundations of Large Language Models

1.1 What LLMs are built on

Large Language Models are based on neural networks and deep learning. A neural network has a vast number of parameters (weights and biases) — often in the billions or trillions — learned during training. LLMs generate text by predicting the next word in a sequence, leveraging **probability** and **conditional probability** (Bayes' theorem): given the words seen so far, the model estimates the probability distribution over the next token and samples from it.

Compared to older architectures like RNNs and LSTMs, **transformers** can process long sequences efficiently and capture long-range dependencies far better. Transformers are the backbone of modern LLMs, and their core innovation is the **attention mechanism** — especially **self-attention** and **multi-head attention** — which lets the model weigh how much each token in a sequence matters to every other token, producing much richer context understanding than RNNs/LSTMs could achieve.

1.2 Tokenization and embeddings

- **Tokenization**: the process of converting raw text into tokens — words, subwords, punctuation, numbers, or symbols. Each token is then assigned a unique numeric ID. Vocabulary size is finite but typically large (tens of thousands of tokens).
- **Embeddings**: tokens are converted from scalar IDs into vectors in a continuous, multi-dimensional space, specifically to capture **semantic meaning**. For example, the word “king” might be represented as a vector like $[0.25, 0.25, \dots]$. Semantically related words end up as nearby vectors in this space.
- **Positional encoding**: added to embeddings to retain the *order* of tokens — this is crucial because word order changes meaning, and the core Transformer/attention math on its own has no inherent sense of sequence position.

1.3 The attention mechanism

Attention computes relationships between tokens by assigning **attention weights** that represent how important one token is relative to the others in the sequence.

- Each token generates three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**.
- A token’s Query is compared against the Keys of all tokens to produce raw attention scores.
- The **softmax** function normalizes these raw scores into probabilities — this is how the model decides how much weight each token’s Value should contribute to the output.
- This mechanism is what lets transformers capture context and semantic relationships far more effectively than RNNs/LSTMs, which process tokens strictly sequentially.

1.4 Training, parameters, and hyperparameters

- LLMs are **pretrained** on vast datasets (books, web pages, code, articles), then **fine-tuned** for specific downstream tasks.
- **Parameters** = the learned weights and biases inside the network, adjusted during training — often numbering in the billions or trillions.
- **Hyperparameters** = configuration values set *before* training (e.g., learning rate, batch size) that influence how training proceeds, but which are not themselves learned by the model.
- Training large models requires enormous computational resources and powerful servers.

1.5 Prompting types (introductory)

- **Zero-shot**: no examples given; the model relies purely on the instruction.
- **One-shot**: exactly one example is given.
- **Few-shot**: multiple examples are given, especially useful when a specific output style or pattern is needed.
- Longer/more complex prompts aren’t inherently “expensive” in a meaningful sense, but they do require careful design to reliably elicit the response you want.

1.6 Limitations of LLMs

- LLMs do not have true understanding or reasoning — every output is a probabilistic prediction based on learned patterns, not a verified fact lookup.
- **Hallucination**: the model produces a confident but incorrect or fabricated answer — including fake citations or invented facts.
- **Bias**: models can reflect biases present in their training data (e.g., political or cultural biases).
- **Privacy and security risks**: sharing personal information with a generative AI system carries exposure risk.
- LLMs have limited real-time knowledge and can produce outdated or incorrect information about recent events.

1.7 Where the field is headed

- Specialized, smaller domain-specific language models (e.g., medical, financial, defense) may outperform generic large models for certain focused tasks.
- Understanding deep-learning fundamentals is valuable for anyone working with LLMs.
- Use LLMs as learning aids to build your own understanding and skill, rather than purely to copy generated text.

Key terms: token, embedding, positional encoding, Query/Key/Value, softmax, self-attention, multi-head attention, parameters vs hyperparameters, pretraining, fine-tuning, hallucination, bias, zero/one/few-shot.

2. Prompt Engineering and Reasoning Strategies

2.1 What prompt engineering is

Prompt engineering is the practice of crafting input instructions or questions in simple, clear language to guide an LLM toward the output you want. Because LLMs predict text token by token based on learned probabilities, precise prompt design directly affects how consistent, accurate, and relevant the output is — this matters even more when you're calling an LLM programmatically through an API inside a larger application.

2.2 Types of prompts

- **System Prompt**: sets the persona, role, or context for the LLM's responses (e.g., "act as a doctor," "act as a backend engineer," "act as a teacher").
- **User Prompt**: the actual question or command posed by the user.
- **Assistant Prompt**: represents the context/memory from previous interactions — carrying forward multi-turn conversation history or other accumulated context.

2.3 The prompt skeleton

A well-structured prompt typically has these components:

Component	Purpose
Role	The persona or expertise the LLM should adopt (e.g., “senior AI developer”)
Context	Background information relevant to the task
Task	The specific action/output expected
Format	Desired output shape — plain text, JSON, table, etc.
Length	Limits on response length, which also manages token usage/cost

Using this structure helps the model understand exactly what’s expected and improves output quality.

2.4 Controlling output with constraints

You can specify **negatives/constraints** — things the model should avoid (no emojis, no rude language, etc.) — to keep outputs professional and on-target.

2.5 Prompt optimization techniques

- **Decomposition**: break a complex task into multiple smaller prompts instead of one giant prompt. Example: separate prompts for script writing, validation, and thumbnail design in a content-creation pipeline, rather than asking for everything in one shot.
- **Chain of Thought (CoT)**: encourage the model to reason step by step rather than jumping straight to an answer — this measurably improves accuracy on complex problems.
- **Few-shot / zero-shot** (recap): zero-shot relies purely on instructions; one-shot/ two-shot/few-shot give one or more examples to steer the response toward a specific pattern or style. Few-shot prompting especially boosts performance when a very specific output style is needed.

2.6 Using LLM APIs in practice

- Interacting with providers like OpenAI or GROQ programmatically involves initializing a client, securely passing an API key, and sending messages that carry a **role** (system/ user/assistant) and **content**.
- Typical uses: definitions, sentiment analysis, product-information extraction, and more.
- Always check the latest documentation for the provider/library you’re using — APIs evolve. Frameworks like **LangChain** offer a unified interface across multiple LLM providers.

2.7 Cost and token management

- Tokens are the chunks of text the API processes for both input and output.
- **Input tokens are generally cheaper; output tokens cost more.**
- Good prompt design balances how many tokens you use against the output quality you need — start small, iterate, then scale prompt size only as needed.

2.8 Best-practice summary

Define role/context/task/format/length clearly, set positive and negative constraints, decompose complex tasks, use chain-of-thought for hard reasoning problems, experiment with few-shot examples, and test prompts repeatedly for consistency (aim for a prompt that reliably scores 9/10 or better). Prompt engineering is iterative — it improves with practice.

Key terms: System/User/Assistant prompt, Role/Context/Task/Format/Length, decomposition, Chain of Thought, zero/one/few-shot, input vs output token cost.

3. Fundamentals of AI Agents and Tool Usage

3.1 From a plain LLM to an agent

An LLM alone is simple: you send a prompt, it generates an output. But LLMs can be enhanced by giving them **tools** (like web search or custom functions) that let them perform additional tasks beyond pure text generation. An **agent** is an LLM combined with tools that can autonomously decide to use those tools to complete complex tasks.

3.2 The ReAct pattern (Reason + Act)

Agents operate on a two-step loop:

1. **Reason** — analyze the task/question and decide what to do next.
2. **Act** — call a tool, or produce the final output.

After acting, the agent re-assesses the result and may repeat the cycle. This is called the “**ReAct**” **pattern** (Reason + Act), and it’s what supports autonomous, iterative decision-making in an agent.

3.3 Creating and describing tools

- A **tool** is a function that performs a specific operation — e.g., a `Calculate` tool (uses Python’s `eval()` to compute an expression) or a `GetWeather` tool (returns weather data from a dataset).
- Tools are described to the LLM in a **JSON format** that explains the tool’s function, its parameters, and a description — this lets the LLM decide when and how to call it.
- When the LLM decides to use a tool, it issues a **tool call** with parameters. External code executes the actual function and returns the result, which the LLM then uses to produce its answer.

3.4 Building a basic agent from scratch (manual loop)

Without relying on any third-party agent framework, the steps are:

1. Initialize the message list with a system prompt and a user prompt.
2. Call the LLM API with the messages and the tool definitions.
3. Check the LLM's output: if it requests a tool call, extract the tool name and parameters.
4. Execute the matching tool function with those parameters.
5. Append the tool's output back into the message list.
6. Repeat the API call with the updated messages until a final answer is produced (i.e., until the LLM stops requesting tools).

Because LLMs process text only, all tool inputs/outputs should be converted to strings before being appended back into the conversation.

3.5 A worked example: data-analysis agent

A simple data-analysis agent can be built around a pandas DataFrame, with tools defined to: list columns, calculate statistical summaries, and count rows matching some criteria. The LLM uses these tools to answer natural-language questions like “what’s the average salary?” or “how many people are in each department?” — this illustrates how domain-specific agents are built around a dataset plus a small set of custom tools.

3.6 Practical exercise flavor

A common practice exercise is building a simple FAQ agent for customer support: it takes user questions (about product issues, delays, return policies, etc.), classifies them as general or product-specific, and answers using a small FAQ data structure combined with the LLM.

Key terms: agent, tool, ReAct (Reason + Act), tool JSON description, manual agent loop, tool call.

4. Structured Outputs and Reliable Agent Workflows

4.1 Why structure matters

LLMs are **probabilistic** — the same query can produce different answers on different calls. This is a problem when chaining multiple agents/tools together, since one agent's output becomes another's input. Enforcing a consistent output format is essential for building robust, chainable multi-agent workflows; small format errors can otherwise cascade and degrade the whole pipeline.

4.2 The temperature parameter

Temperature is a hyperparameter controlling the randomness of an LLM's output:

- **Temperature = 0** → deterministic, focused responses with minimal creativity — ideal for structured data extraction, where you want low output variance.

- **Higher temperature** → more creativity and variability — better suited to creative tasks.
- Some newer reasoning models use an “effort” parameter instead of temperature to control this trade-off.

4.3 Enforcing structured output with JSON Schema

JSON is the primary interchange format for structured data in LLM-based agents. A **JSON schema** lets you specify:

- Required fields (e.g., product name, price per unit, quantity, total cost)
- Data types for each field (string, number, etc.)
- Strict enforcement, so the model can’t silently produce unexpected fields

This reduces the risk of missing fields or unexpected structural drift in the output.

4.4 Output validation

Even with a schema defined, an LLM’s probabilistic output might not always be valid JSON or fully conform to the schema, so validation is still necessary before passing data downstream:

- **JSON parsing** — is it even valid JSON?
- **Schema compliance** — are the required keys present, with the right data types?
- **Logical checks** — e.g., price and quantity shouldn’t be zero for a real order.

If validation fails, an **error feedback loop** can ask the LLM to produce a corrected output.

4.5 Structured tool outputs

Tools integrated with agents should return **structured JSON**, not unstructured strings. Example: a weather-API tool returns {city, condition, temperature} as JSON so downstream agents can reliably consume the result. On error (missing/invalid parameters), tools should return a structured error response, not an unstructured message that could break the calling code.

4.6 Idempotent tool design

An **idempotent** tool produces the same overall result if called multiple times with the same inputs, without additional side-effects from the repeated calls. This matters for tools that might get called more than once by accident (e.g., a retry after a timeout).

- **Non-idempotent example:** a payment-processing tool that deducts money on every call — call it twice by accident, and the user is charged twice.
- **Fix:** use a **request ID** to detect and ignore duplicate calls, so a repeated call with the same request ID has no additional effect.

4.7 Multi-agent orchestration and consistency

Complex workflows chain multiple agents and tools together. Enforcing structured outputs and validation at every step is what makes this kind of orchestration reliable

— agents decide which tools to invoke, pass outputs downstream, and consistency/ idempotency prevent a single failure from propagating through the whole pipeline.

Key terms: temperature, JSON schema, output validation, error feedback loop, structured tool output, idempotent tool, request ID.

5. Embeddings and Semantic Retrieval

5.1 From one-hot encoding to embeddings

Traditional **one-hot encoding** converts text into numeric form but captures no semantic meaning at all — every word is just an arbitrary, equally-distant index. **Embeddings** instead convert text into dense numeric vectors that capture semantic meaning: words like “hi” and “hello” end up close together in vector space because they mean similar things. This is what powers **semantic search** — matching a query to documents by *meaning*, even when the exact keywords don’t match.

5.2 How embeddings are produced

Text is tokenized, then transformed into a high-dimensional vector (common sizes: 512, 1536 dimensions). The distance between two vectors indicates semantic similarity — closer points mean more similar meaning. Embedding models are typically built on transformer architecture using self-attention (the same “Attention Is All You Need” foundation covered in Section 1). Hugging Face hosts many open-source embedding models; OpenAI also provides embedding models via API (e.g., `text-embedding-3-small`, which produces 1536-dimensional vectors). Companies often prefer cloud-hosted embedding APIs for trust, quality, and support, even though open-source alternatives exist.

5.3 Cosine similarity

Cosine similarity measures the angle between two vectors to quantify similarity, producing a score between 0 and 1:

- **Score > 0.8** → strong semantic similarity
- **Score 0.5–0.8** → related, but only loosely similar
- **Score < 0.5** → unrelated content

Formula: $\text{cosine similarity} = \frac{\text{dot product of the two vectors}}{\text{product of their norms (magnitudes)}}$. These specific numeric cutoffs are a useful convention, not a universal law — different embedding models produce different score distributions, so real systems calibrate thresholds empirically rather than assuming fixed numbers hold everywhere. Also note: similarity doesn’t always track factual correctness — training-data biases can make contradictory statements score as “similar.”

5.4 Chunking for scalable search

Large documents are split into **chunks** (e.g., paragraph-level) before embedding, since loading and embedding an entire document as one unit doesn’t scale. Each

chunk is embedded and stored as its own vector (with metadata like chunk index/source document). At query time, the query itself is embedded, and similarity is computed against every stored chunk embedding — the top-matching chunks are retrieved as the relevant context. This keeps search efficient even over very large document collections.

5.5 Vector databases

Storing embeddings in a traditional database is inefficient for similarity search because of the high dimensionality and volume of vectors involved. **Vector databases** (e.g., **ChromaDB**, **Faiss**, **Milvus**, **Weaviate**, **Pinecone**) are purpose-built to index and search high-dimensional vectors efficiently, alongside their metadata. They typically use specialized nearest-neighbor search algorithms such as **HNSW** (Hierarchical Navigable Small World graphs) to speed up retrieval at scale.

ChromaDB specifically is a popular open-source vector database that handles embedding storage, indexing, and similarity search internally — it can even convert text directly into embeddings for you, removing some manual preprocessing steps. It supports adding chunks + embeddings + metadata together, then querying for the most similar stored chunks.

Key terms: one-hot encoding, embedding, cosine similarity, chunking, vector database, ChromaDB, HNSW.

6. Retrieval-Augmented Generation (RAG)

6.1 What RAG is and why it exists

RAG (Retrieval-Augmented Generation) combines retrieving relevant chunks from a document database with an LLM's generation capability, so the model can answer questions about specific — often private — documents it was never trained on. By grounding answers strictly in retrieved data, RAG substantially mitigates **hallucination** compared to querying a general LLM directly.

Challenges RAG addresses:

- LLMs have no built-in access to private/internal documents unless explicitly provided.
- Large documents (or many documents) don't fit inside an LLM's limited context window.
- Sending large volumes of text to a hosted API repeatedly is costly (tokens + latency).
- Handling sensitive information securely, without leaking it, is critical.
- Solutions need to scale as the number of documents or users grows.

6.2 The two-phase RAG architecture

1. Indexing Phase (one-time setup): - Collect documents and split them into chunks (paragraph-level, line-level, semantic chunking, fixed-size, or sliding-window).

- Convert each chunk into a vector embedding (e.g., using a model like `all-MiniLM-L6-v2`). - Store the embeddings, along with metadata (source document, chunk info), in a **vector database** such as ChromaDB.

2. Retrieval Phase (query time): - Embed the user's query using the *same* embedding model used during indexing. - Retrieve the **top-k** most similar chunks from the vector database (via cosine similarity or another similarity metric). - Construct a prompt that combines the retrieved chunks as context with the user's query — with explicit instructions to answer *only* from that context, and to say “I don't know” if the context is insufficient. - The LLM generates a grounded, factual answer, traceable back to the source chunks.

6.3 Practical implementation notes

A minimal RAG pipeline: read documents → split into chunks (e.g., by paragraph) → create embeddings for each chunk and store them (with metadata) in ChromaDB → embed the incoming query and retrieve the top-k similar chunks → build a prompt with the retrieved context + the query → send to an LLM (e.g., GPT-4o-mini) to generate the grounded answer. The underlying code for a basic RAG system is fairly simple — the real complexity is architectural and in tuning data handling, not the code itself.

6.4 Beyond naive RAG

Beyond the basic (“naive”) RAG pattern above, there are more advanced variants — graph RAG, agentic RAG, modular RAG, and multi-modal RAG (handling images, tables, video) — aimed at improving complexity handling, speed, scalability, and accuracy for harder use cases.

Cache-Augmented Generation (CAG), as introduced alongside RAG here, refers to adding a caching layer that stores question-answer pairs for repeated or similar queries, so the system can skip redundant retrieval work on repeat questions.

6.5 Evaluating and troubleshooting RAG systems

- **Evaluation approaches:** manual dataset-based evaluation using question-answer pairs; metrics like cosine similarity, precision, and recall; practical checks on latency and reliability.
- **Common challenges:** preserving semantic coherence when choosing a chunk size/strategy; avoiding partial or fragmented information loss; managing hallucination and “cold start” problems; balancing how many chunks you retrieve (top-k) against the LLM's token limit; and handling large-scale document collections efficiently.
- **Practical advice:** build a RAG system from scratch at least once to understand the moving parts before relying on high-level libraries (e.g., LangChain). Production-grade RAG systems can take months to get right because of data-handling, tuning, and integration complexity. Always tune retrieval parameters (top-k, chunk size/strategy) to your specific data and use case. Guardrails and careful prompt engineering are essential for safe, reliable real-world deployment.

Key terms: RAG, indexing phase, retrieval phase, top-k retrieval, grounded answer, graph/agentic/modular/multi-modal RAG, Cache-Augmented Generation.

7. Productionizing LLM Applications with FastAPI

7.1 Why turn a notebook prototype into an API

A RAG chatbot (or any LLM application) built and tested locally (e.g., in Google Colab) needs to become an **API** so that other applications or users can interact with it without needing your local development environment.

7.2 FastAPI basics

FastAPI is a Python library that exposes ordinary functions as HTTP API endpoints. It supports fast development, automatic interactive documentation (Swagger UI at `/docs`), and built-in input validation.

- `@app.get` — defines a GET endpoint; parameters are typically passed via the URL query string (e.g., `?A=10&B=25`).
- `@app.post` — defines a POST endpoint; typically takes a JSON request body, validated by a Pydantic model.
- **Uvicorn** is the ASGI server used to actually run a FastAPI application and listen for HTTP requests (e.g., `uvicorn main:app --reload` for local development with hot reload).

7.3 Input validation with Pydantic

Plain Python functions don't validate their inputs — you could pass a string where an integer is expected and get a confusing runtime error deep in your code. **Pydantic** solves this: you define a `BaseModel` subclass specifying the expected fields and their types, and FastAPI uses that model to validate incoming request payloads automatically, returning clear, descriptive errors when the input doesn't match.

7.4 Building a RAG chatbot API

Practical shape of the API:

- A health-check endpoint (e.g., `/`).
- A `/chat` endpoint that accepts a question, runs it through the retriever + LLM (as described in Section 6), and returns a response — typically structured as `{answer, source_documents, request_id}` using Pydantic request/response models.
- **Logging** (via Python's logging module, not scattered `print()` statements) adds observability for debugging and monitoring.
- Test locally with Uvicorn, and explore/exercise the endpoints via the auto-generated Swagger UI at `/docs`.

7.5 Deploying to Hugging Face Spaces

Hugging Face Spaces is a free hosting platform for ML applications/APIs, supporting public deployment via Docker.

Deployment steps:

1. Create a Hugging Face account and a Space repository.
2. Clone the Space repo locally with git.
3. Copy your project files (source code, `requirements.txt`, `Dockerfile`, any data assets) into the repo.
4. Add sensitive values (e.g., your LLM API key, HF token) as **secrets** in the Space's settings — never commit them directly into the repo.
5. Commit and push the code — this triggers Hugging Face to build and deploy the Docker container automatically.
6. The application becomes publicly accessible at a Hugging Face-provided URL.

The **Dockerfile** defines the environment: base Python image, working directory, dependency installation, and the start command (which runs Uvicorn inside the container, alongside the packaged code and any needed data assets).

7.6 Odds and ends

A frontend (e.g., built with **Streamlit**) can be layered on top of a FastAPI backend if you want a quick UI — deployment/Docker considerations apply similarly in that case.

Key terms: FastAPI, `@app.get` / `@app.post`, Uvicorn, Pydantic `BaseModel`, `Dockerfile`, Hugging Face Spaces secrets.

8. Orchestration and Agent Workflow Design with Langgraph

8.1 From a simple loop to a graph

A single agent can be built with a `while` loop that repeatedly calls an LLM and checks whether the response requires a tool call or a stop — this works for simple, sequential flows. But real-world workflows often need multiple tools, branching decision paths, and coordination between multiple agents — a simple loop doesn't scale to that complexity.

8.2 Langgraph core concepts

Langgraph is a framework for building graph-structured agent workflows, supporting multiple nodes, edges, branching, conditional routing, and shared state.

Concept	Meaning
State Graph	The overall workflow definition
Node	An individual step or function in the workflow
Edge	A connection between nodes — conditional or unconditional

Concept	Meaning
Conditional Edge	An edge whose destination depends on a decision/condition
State	Shared data flowing through the graph's nodes — like a shared dictionary/clipboard that every node can read and update

Benefits: persistence through failures, support for human-in-the-loop steps, observability (e.g., via **Langsmith**), and the ability to compose complex multi-agent orchestrations suitable for production.

8.3 Typing the shared state: TypedDict vs Pydantic

Plain Python dictionaries are fully dynamic — any key, any value type — which invites subtle bugs in a complex workflow.

- **TypedDict** lets you statically declare the expected keys and types of a dictionary, improving clarity and editor/type-checking support — but it does **not** enforce anything at runtime.
- **Pydantic's BaseModel** enforces strict *runtime* type validation. This is powerful, but it conflicts with the incremental, partial state updates that are common in Langraph workflows (where one node might update just one field of the shared state at a time).
- **Why TypedDict is generally preferred for Langraph state:** it allows incremental, partial updates without rejecting them, since it isn't enforcing anything at runtime — which fits how state naturally flows through a graph of nodes better than Pydantic's strict validation does.

8.4 Practical Langraph code shape

```
from typing import TypedDict

# Define a state class to type the shared state between nodes
class State(TypedDict):
    question: str
    answer: str

# A node is just a function that reads/updates the shared state
def greet(state: State) -> State:
    state['answer'] = f"Hello {state['question']}! Welcome to Langraph tutorial."
    return state

# Build the graph: add nodes and edges, then compile before running
graph = StateGraph(state_type=State)
graph.add_node('greet', greet)
graph.add_edge('start', 'greet')
```

```
graph.add_edge('greet', 'end')
app = graph.compile()

# Run it
result = app.invoke({'question': "Balaji"})
print(result['answer']) # Output: Hello Balaji! Welcome to Langraph tutorial.
```

A node can also call an LLM directly — e.g., a node that classifies the intent of an incoming question:

```
def analyze_intent(state: State) -> State:
    topic = call_llm(
        prompt=f"Classify this question into one of billing, HR, general: {state['question']}",
        system_prompt="You classify questions into exactly one category keyword."
    )
    state['topic'] = topic.strip()
    return state

# Conditional routing based on the classified intent
graph.add_conditional_edge('classify', 'billing_handler', condition=lambda s:
s['topic'] == 'billing')
graph.add_conditional_edge('classify', 'hr_handler', condition=lambda s:
s['topic'] == 'hr')
graph.add_conditional_edge('classify', 'general_handler', condition=lambda s:
s['topic'] == 'general')
```

8.5 Conditional and parallel workflows

A common pattern: a **classifier node** determines a question's intent (e.g., Billing, HR, General), and **conditional edges** route the flow to the matching handler node, each of which processes its intent and produces an answer. Langraph supports graphs that go well beyond simple linear flows — including parallel branches and more complex routing logic — and agents built this way can have their own sub-workflows that compose into larger, multi-agent systems with shared state and inter-agent communication.

8.6 RAG inside a graph

A vector store (e.g., ChromaDB) can be integrated directly into a Langraph node, so a query gets routed to the right handler (e.g., billing or HR) and that handler retrieves relevant documents from the vector store to answer using the RAG pattern from Section 6 — combining orchestration with retrieval-grounded answers in one workflow.

8.7 Supporting tools and ongoing learning

- **Langsmith**: an observability/debugging tool for Langraph workflows.
- **Skills** and **MCPs** (multi-capability plugins) are additional concepts for automating external app integrations and recurring tasks.

- **MLflow** is a tool commonly recommended for tracking ML experiments and workflows.
- Continuous learning — of new tools, frameworks, and concepts — is essential in this fast-moving space; practical, hands-on experience matters more than any single certification.

Key terms: Langgraph, State Graph, Node, Edge, Conditional Edge, shared State, TypedDict vs Pydantic BaseModel, .compile() / .invoke(), intent classification routing, Langsmith.

Coverage checklist

This guide covers every in-syllabus subtopic identified for this evaluation, plus additional context (mentioned briefly) for a fuller picture: named commercial LLM platforms, brief live-coding recaps, Colab/API-key setup tips, the FAQ-agent practice exercise, advanced RAG variants (graph/agentive/modular/multi-modal — named at a high level above), Streamlit as an optional frontend, and Langsmith/Skills-MCPs/MLflow as supporting-tool mentions. If a concept from your lectures isn't listed above, it was likely a very brief, name-only mention rather than something requiring its own section here — but everything actively taught across all eight sessions is represented.